



Cuestionario Teórico

Grupo 2

Técnicas Digitales III

Curso 512 - 2025

Iván Gonzalo Irumberry

Matwiejczuk, Leandro Darío

Profesor: Ing. MARÍN Ariel

Auxiliar: CARLASSARA Fabrizio

Auxiliar: VAN ROEY Julián

Cuestionario - Preguntas

1) Sistemas Operativos - Generalidades

- a) ¿Quién es el encargado de administrar el tiempo de la CPU y cual es su función principal dentro del sistema?
- b) Explique a que se denomina “Contexto de Ejecución”. ¿Cuáles son las variables intervinientes?
- c) ¿A que se denomina Sistema Operativo de Tiempo Real? ¿Por qué usar un Sistema Operativo de Tiempo Real?

2) Kernel de FreeRTOS

- a) Explique y desarrolle las características de una “Tarea” en FreeRTOS. Proponga un ejemplo de una tarea manteniendo la sintaxis correcta.
- b) Explique y desarrolle los “Estados de una Tarea” y sus transiciones. Indique las funciones que permiten las transiciones.
- c) ¿A qué nos referimos con el concepto de “tareas de procesamiento continuo”? ¿Y a qué nos referimos con el concepto de “tareas manejadas por eventos”?
- d) ¿Cuáles son los tipos de eventos por los que una tarea puede estar esperando al entrar en el estado bloqueado? Explicar.
- e) ¿Cuál es la función de la IDLE TASK? ¿Puede el procesador no estar ejecutando ninguna tarea en algún momento? Explicar.

3) Desarrolle como es el almacenamiento de datos en el recurso “Cola” proporcionado por el Kernel.

4) Explique utilizando un ejemplo con código el porqué es común que el recurso Cola posea múltiples tareas escritoras y una sola tarea lectora. Explique el proceso de bloqueo por escritura y bloqueo por lectura en el recurso Cola.

5) A la hora de enviar datos compuestos: explique cómo se lleva a cabo dicho proceso y por que no genera conflictos con la definición de la macro del recurso. ¿Cuál sería la implementación si los datos fueran “grandes”, lo que ocasiona un problema con la memoria RAM?

6) Explicar el funcionamiento del esquema de la fig 1:

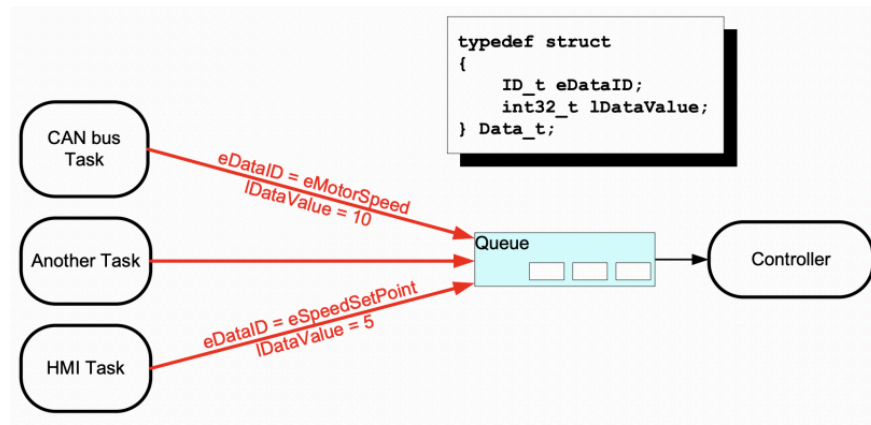


Fig. 1. Recibir datos de múltiples fuentes

- 7) Desarrollar un programa ejemplo que permita comprender el concepto y uso de “Semáforos Mutex”.
- 8) Desarrollar un programa ejemplo que permita comprender el concepto y uso de “Inversión de Prioridad”. Como mínimo se deben utilizar dos tareas y una cola gestionando un recurso del microcontrolador.
- 9) Explicar el concepto de “Herencia de Prioridad” utilizando diagramas gráficos que permitan comprenderlo. ’
- 10) Desarrollar un programa ejemplo que permita comprender el concepto de “Punto Muerto”. Como mínimo se deben utilizar dos tareas y una cola gestionando un recurso del microcontrolador.

Cuestionario - Respuestas

1)

- a) El encargado de administrar el tiempo de la CPU en un sist. operativo es el scheduler, el cuál se encarga de controlar y administrar los recursos del sistema para que cada tarea se ejecute “en tiempo real” y así poder tener un sistema multitarea. En un sistema multitarea el tiempo de ejecución se reparte entre cada tarea para simular la ejecución de tareas al mismo tiempo.
- b) El contexto de ejecución es toda la información sobre la ejecución de una tarea, como por ejemplo los registros del microcontrolador sobre el cuál ejecuta la tarea o con los que interactúa.
- c) Un sistema operativo de tiempo real es un sistema operativo liviano utilizado para desarrollar cosas íntimas e integrar tareas de diseño con recursos y tiempos específicos de manera óptima y sencilla.
Se utiliza un sistema operativo de tiempo real cuando se necesita una sincronización entre tareas, que se regulan utilizando los recursos y APIs de dicho sistema operativo.

2)

- a) Una tarea en FreeRTOS se define como una función en C, la cual posee características que la diferencian de ser una simple función en C.
Estas características son:
 - Nunca debe terminar: Debe ejecutarse un bucle infinito dentro de esta función. Si ya no se requiere su ejecución, se debe borrar.
 - Sistema de prioridad: Cada tarea posee una prioridad asociada, si la prioridad de una tarea A es mayor a la de una tarea B y ambas se encuentran siempre en un estado que permite ejecutarlas, siempre se ejecutará tarea A por ser de mayor prioridad.
 - Tamaño en stack: Cada tarea posee un espacio de memoria del dispositivo alojado en el sistema, donde se guardan, por ejemplo, sus variables locales.
 - Valor de retorno nulo: Como nunca termina, no debe devolver un valor.
 - Puntero a void como parámetro: El puntero a void permitirá enviar cualquier tipo de dato como parámetro a la tarea creada.

Ejemplo:

```

1 /* Defino la tarea */
2 void Task(void *param)
3 {
4     for(;;); // Bucle infinito, la tarea nunca debe terminar
5 }
6
7 int main(void)
8 {
9     /* Creo la tarea */
10    xTaskCreate(Task, /* Puntero a la función a implementar */
11               "Task 1", /* Texto para facilitar el debugging */
12               1000, /* Tamaño en pila */
13               NULL, /* Parámetro que recibirá la tarea, en este caso nada */
14               1, /* Prioridad de la tarea */
15               NULL); /* Task handler, en este caso nulo */
16 /* Inicializo el scheduler */
17 vTaskStartScheduler();
18 }

```

b) Los estados de una tarea pueden ser:

- Running: La tarea está siendo ejecutada por el procesador.
- Ready: La tarea está lista para ser ejecutada por el procesador. Al generarse un cambio de contexto, si tiene la suficiente prioridad, pasará a ser ejecutada.
- Bloqueada: La tarea se encuentra a la espera de algún evento para poder continuar con su ejecución. Dicho evento puede ser un evento temporal (retardo de tiempo que debe cumplirse) o un evento de sincronización, el cual es originado desde otra tarea o interrupción.
- Suspendida: La tarea no está disponible para que el scheduler la ejecute. La única manera de que entre en este estado es llamando a la función `vTaskSuspend()` y la única manera de salir de este estado es llamando a `vTaskResume()` o `vTaskResumeFromISR()`.

Las tareas transcurren por los distintos estados luego de llamar a funciones API (*application-program interface*).

Las API permiten manejar recursos del sistema operativo, y por ende el bloqueo, desbloqueo, suspensión o reanudación de las tareas.

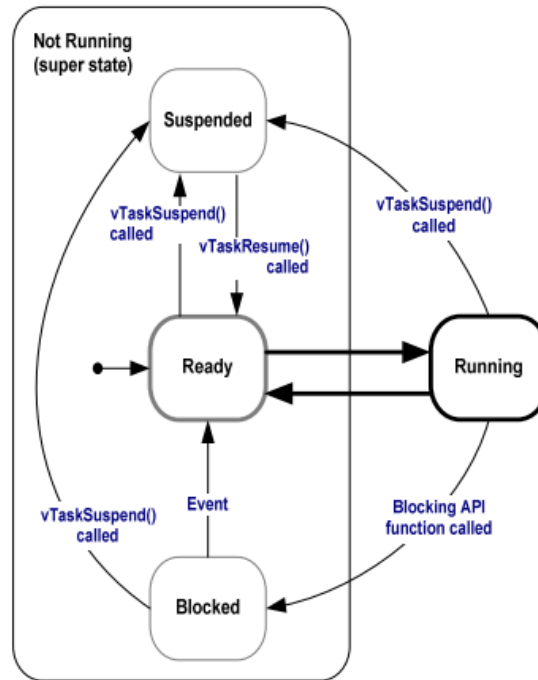


Figure 7 Full task state machine

- c) Las tareas de procesamiento continuo son aquellas que siempre están en el estado “Ready” o siempre están ejecutándose. Estas tareas deberían ser creadas con una prioridad baja ya que, al tener un funcionamiento continuo, no permiten que tareas de menor prioridad se ejecuten.

Las tareas manejadas por eventos son aquellas que se bloquean esperando a que algo suceda para resumir su ejecución (por ejemplo esperar un semáforo, cola, etc.).

- d) Los tipos de evento por los cuáles una tarea puede estar esperando al estar en el estado bloqueado pueden ser la utilización de un recurso compartido por parte de otra tarea, la lectura o escritura en una cola vacía o llena, un semáforo, bloqueos por demoras de tiempo, etc. Esto sirve para sincronizar el funcionamiento del sistema, aprovechando el uso de las prioridades, que caracterizan un sistema multitarea.
- e) La IDLE TASK (tarea ociosa en español) es la tarea de menor prioridad del sistema operativo. Ésta se crea de manera automática al crear el scheduler para permitir que siempre se esté ejecutando una tarea. La IDLE TASK es un bucle infinito que termina su ejecución cuando el scheduler determina que hay alguna otra tarea en estado Ready. Además, es la responsable de liberar memoria alojada por el sistema operativo en caso de que una tarea se haya borrado utilizando `vTaskDelete()`.

Se puede brindar funcionalidad a la tarea ociosa mediante los “idle hooks” que son funciones ejecutadas cada ciclo de la tarea, esto es útil para, por ejemplo, colocar al CPU en un estado de ahorro energético, entre otros usos.

3)

Una cola puede contener una cantidad finita de elementos, elementos cuyo tipo debe ser declarado con antelación. También, se debe declarar la longitud de la cola (cantidad de elementos que podrá almacenar). Todos los elementos de una cola deben ser iguales, por ejemplo, una cola de diez elementos de tipo entero.

Las colas se usan generalmente como buffers entre tareas de tipo first input first output (FIFO) donde se puede escribir tanto al inicio como al final de la cola con las APIs `xQueueSendToFront()` y `xQueueSendToBack()`.

Todos los datos en una cola son almacenados por copia, lo que puede causar un gran uso de memoria si se las utiliza sin cuidado.

Al leer los datos de una cola, el valor leído se elimina de la cola.

4)

Es usual que una cola tenga más de una tarea escritora y una tarea lectora ya que desde la tarea lectora (guardiana) se puede bloquear una tarea para que otra se desbloquee así controlando el flujo de datos de manera segura.

Por ejemplo, dos tareas con igual prioridad sensan el estado de dos botones y una tarea guardiana de mayor prioridad se desbloquea leyendo una cola compartida de un elemento, esta cola puede ser escrita por cualquiera de las tareas que revisa los botones, pero este funcionamiento asegura la seguridad del programa (por ejemplo qué pasa si tarea boton 1 envía un dato y mientras se envía se presiona el botón 2 de tarea boton 2).

```
1 /* Handler de la cola */
2 QueueHandle_t queue;
3 /* Tarea boton 1 que envia el dato a la cola */
4 void boton1(void *pv)
5 {
6     uint32_t boton_state;
7     for(;;)
8     {
9         /* Leer el estado del botón 1 */
10         if(HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0) == GPIO_PIN_RESET)
11             boton_state = 1;
12         else
13             boton_state = 0;
14         /* Sube el dato a la cola */
15         xQueueSendToBack(queue, &boton_state, portMAX_DELAY);
16         vTaskDelay(50/portTICK_PERIOD_MS);
17     }
18 }
19 /* Tarea boton 2 que envia el dato a la cola */
20 void boton2(void *pv)
21 {
22     uint32_t boton_state;
23     for(;;)
24     {
25         /* Leer el estado del botón 2 */
26         if(HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_1) == GPIO_PIN_RESET)
27             boton_state = 2;
28         else
```

```
29         boton_state = 3;
30     /* Sube el dato a la cola */
31     xQueueSendToBack(queue, &boton_state, portMAX_DELAY);
32     vTaskDelay(50/portTICK_PERIOD_MS);
33 }
34 }
35 /* Tarea controladora */
36 void Control(void *pv)
37 {
38     uint16_t boton_state;
39     for(;;)
40     {
41         /* Recibe el dato de la cola */
42         xQueueReceive(queue, &boton_state, portMAX_DELAY);
43         /* Control de los LEDs según el estado de los botones */
44         if(boton_state == 0)
45             /* Hago algo */
46         if(boton_state == 1)
47             /* Hago algo */
48         if(boton_state == 2)
49             /* Hago algo */
50         if(boton_state == 3)
51             /* Hago algo */
52     }
53 }
54 }
55 int main(void)
56 {
57     /* Inicializa la HAL Library */
58     HAL_Init();
59     /* Configuración del reloj del sistema */
60     SystemClock_Config();
61     /* Crear la cola */
62     queue = xQueueCreate(16, sizeof(uint32_t));
63     /* Crear las tareas */
64     xTaskCreate(estadoboton1, "EstadoBoton1", configMINIMAL_STACK_SIZE, NULL, 1,
65     NULL);
66     xTaskCreate(estadoboton2, "EstadoBoton2", configMINIMAL_STACK_SIZE, NULL, 1,
67     NULL);
68     xTaskCreate(Control, "Control", configMINIMAL_STACK_SIZE, NULL, 1, NULL);
69     /* Iniciar el scheduler */
70     vTaskStartScheduler();
71     while(1){}
72 }
```

Bloqueo en lectura de cola:

Cuando desde una tarea se intenta leer una cola, si la misma se encuentra vacía, la tarea se bloqueará por un tiempo determinado por el valor `TickType_t xTicksToWait` que es el tercer parámetro a colocar al utilizar `xQueueReceive`, si el valor `xTicksToWait` es `portMAX_DELAY`, la tarea pasará a estado bloqueado sin límite de tiempo.

Una tarea en estado bloqueado pasará automáticamente a estado Ready cuando alguna interrupción o tarea inserte un valor a la cola.

Dado que las colas pueden tener múltiples lectores, en caso de que varias tareas en estado bloqueado estén esperando a que se introduzca un valor a la cola para desbloquearse tendrá prioridad la tarea con mayor prioridad disponible.

Bloqueo en lectura de cola:

Cuando se escribe en una cola, la tarea puede especificar cuánto tiempo debe estar la tarea en estado bloqueado mientras espera a que otro hilo de ejecución libere el espacio de la cola que estaba llena mediante el tiempo de bloqueo, tercer parámetro de `xQueueSendToBack`.

Las colas pueden tener múltiples escritores por lo que es posible que una cola llena tenga más de una tarea bloqueada a la espera de poder completar una operación de envío. Cuando este sea el caso sólo una tarea se desbloqueará cuando haya espacio disponible en la cola. La tarea que se desbloquea siempre será la tarea más alta prioridad que estaba esperando por el espacio. Si las tareas bloqueadas tienen la misma prioridad entonces se desbloqueará la tarea que más tiempo ha estado esperando a que se libere el espacio.

5)

Es común tener que enviar distintos tipos de datos en una misma cola, dado que enviar desde una misma tarea dos datos es poco práctico debido a que no aprovecha las ventajas del FreeRtos; dentro de una cola todos los elementos deben ser del mismo tipo, una forma de realizar el envío de distintos tipos es mediante estructuras donde dentro de ellas se puede tener distintos tipos de datos.

Si los datos fueran “grandes”, dado que el espacio disponible en RAM es finito, se pueden generar problemas en el funcionamiento del programa si es que no se encuentra espacio para alojar la cola. Una solución sería que la cola no trabaje con estructuras, sino con punteros a estructuras lo que reduciría considerablemente su tamaño.

6)

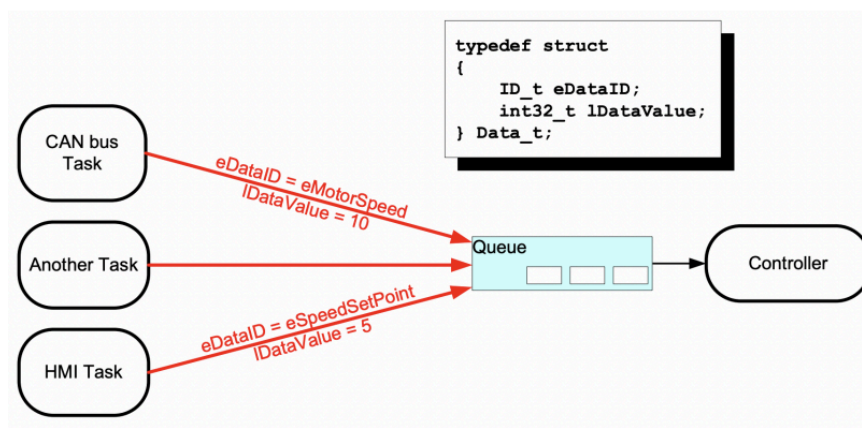


Fig. 1. Recibir datos de múltiples fuentes

Se puede observar en el diagrama como se poseen 3 tareas escritoras a una cola, siendo éstas “CAN bus Task”, “Another Task”, y “HMI Task”; y una tarea lectora “Controller”.

La queue posee 3 elementos de tipo xData, siendo xData una estructura que contiene dos enteros iValue e iMeaning.

Se puede ver como la tarea “CAN bus Task” envía un valor 10 en iValue y una macro MOTOR_SPEED en iMeaning, y cómo la tarea “HMI Task” 5 y NEW_SET_POINT; en este programa la cola se ocupa de recibir los valores numéricos junto con sus macros que luego es enviado a “Controller” que evaluará qué hacer con los valores basándose en la macro que reciba en iMeaning.

Pese a que se observa que la tarea “Another Task” está interactuando con la cola, el diagrama no especifica cómo ni con qué valores lo hace, por lo que se asume que esta tarea nunca envía ningún elemento a la cola.

7) Programa ejemplo semáforos mutex:

```
1  /* Tarea 1 */
2  void vTask1(void *pv)
3  {
4      for(;;)
5      {
6          /* Intentar tomar el mutex */
7          if (xSemaphoreTake(xMutex, portMAX_DELAY) == pdTRUE)
8          {
9              /* acceder al recurso compartido o realizar una acción*/
10             vTaskDelay(500);
11             /* Liberar el mutex */
12             xSemaphoreGive(xMutex);
13         }
14         vTaskDelay(1000);
15     }
16 }
17 /* Tarea 2 */
18 void vTask2(void *pv)
19 {
20     for(;;)
21     {
22         /* Intentar tomar el mutex */
23         if (xSemaphoreTake(xMutex, portMAX_DELAY) == pdTRUE)
24         {
25             /* acceder al recurso compartido o realizar una acción*/
26             vTaskDelay(500)
27             /* Liberar el mutex */
28             xSemaphoreGive(xMutex);
29         }
30         vTaskDelay(1000);
31     }
32 }
```

En este programa se puede observar cómo el semáforo mutex protege recursos compartidos a ambas tareas en secciones denominadas sección crítica de ser escritos o utilizados por dos tareas “a la vez”.

8) Programa ejemplo inversión de prioridad:

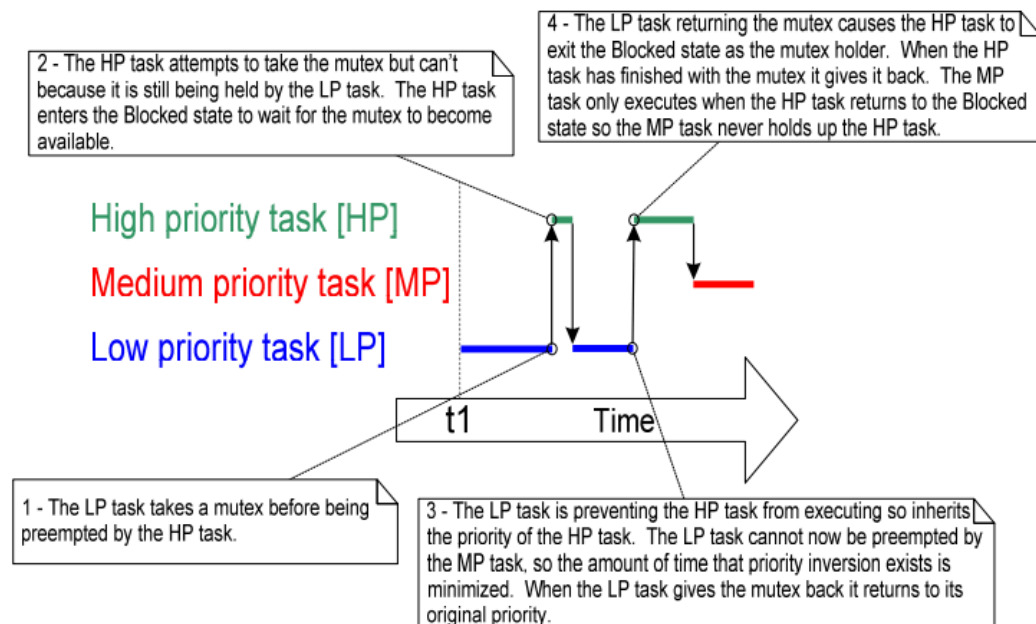
```
1  /* Tarea 1 (Menor Prioridad): Envía comandos a la cola */
2  void vTask1(void *pv)
3  {
4      uint32_t LED;
5      for(;;)
6      {
7          /* Toma el mutex */
8          if (xSemaphoreTake(xMutex, portMAX_DELAY) == pdTRUE)
9          {
10             /* Enviar comando a la cola */
11             LED = 0; // Para apagar el LED
12             xQueueSend(xQueue, &LED, portMAX_DELAY);
13             /* Liberar el mutex */
14             xSemaphoreGive(xMutex);
15             }
16             vTaskDelay(2000);
17         }
18     }
19 /* Tarea 2 (Mayor Prioridad): Enviar comandos a la cola */
20 void vTask2(void *pv)
21 {
22     uint32_t LED ;
23     for(;;)
24     {
25         /* Toma el mutex */
26         if (xSemaphoreTake(xMutex, portMAX_DELAY) == pdTRUE)
27         {
28             /* Enviar comando a la cola */
29             LED = 1; // Para encender el LED
30             xQueueSend(xQueue, &LED, portMAX_DELAY);
31             /* Liberar el mutex */
32             xSemaphoreGive(xMutex);
33         }
34         vTaskDelay(500);
35     }
36 }
37 /* Tarea de Control: Recibe comandos de la cola y controla el LED */
38 void vControlTask(void *pv)
39 {
40     uint32_t LED;
41     for(;;)
42     {
43         /* Recibir comandos de la cola */
44         if (xQueueReceive(xQueue, &command, portMAX_DELAY) == pdPASS)
45         {
46             /* Ejecutar el comando recibido */
47             if (LED == 1)
48             {
49                 /* Encender el LED */
50             }
51             else
52             {
53                 /* Apagar el LED */
54             }
55         }
56     }
57 }
```

En este programa se puede observar como la tarea vControlTask espera recibir datos de la cola llena por vTask1 y vTask2 para encender o apagar un LED. Debido a que la tarea

vTask2 tiene mayor prioridad a vTask1 y tiene un tiempo de bloqueo de 500mS (contra 2000mS de vTask2), la tarea vTask2 se encuentra a la espera de vTask1. Cuando vTask1 se termine de ejecutar liberará el semáforo que será tomado por vTask2. Ambas tareas intentan escribir en una cola que recibe vControlTask.

Se produce una “inversión de prioridad” porque la tarea de mayor prioridad (vTask2) se encuentra a la espera de que la tarea de menor prioridad (vTask1) termine de ejecutarse para liberar el semáforo.

- 9) La herencia de prioridad es cuándo la tarea de menor prioridad tiene el semáforo mutex tomado y en ese momento, si dicho semáforo es compartido con una tarea de mayor prioridad, la prioridad de la tarea menor aumenta circunstancialmente hasta ser la de la tarea mayor. De esta forma se reducen las probabilidades de causar un problema de inversión de prioridades en el programa.



En este gráfico se observa cómo la tarea de baja prioridad toma el semáforo mutex, luego la tarea de alta prioridad intenta tomarlo pero falla al estar ya tomado el semáforo, por lo que se bloquea. Entonces seguiría corriendo la tarea de baja prioridad pero si por algún motivo una tarea de prioridad media empezará a ejecutarse, nunca se reactivaría la tarea de alta prioridad.

Este no es el caso, debido a que la tarea de baja prioridad “hereda” la alta prioridad por lo que la tarea de prioridad media no podrá ejecutarse, esto resuelve el problema de inversión de prioridad.

Cuando la tarea de baja prioridad (ahora alta por la prioridad heredada) termine de ejecutarse liberará el semáforo y el sistema podrá seguir funcionando normalmente.

10) Ejemplo de punto muerto:

```
1  /* Tarea 1 (Menor Prioridad): Envía comandos a la cola */
2  void vTask1(void *pv)
3  {
4      uint32_t LED;
5      for(;;)
6      {
7          /* Toma el mutex x */
8          if (xSemaphoreTake(xMutex, portMAX_DELAY) == pdTRUE)
9          {
10             if (xSemaphoreTake(yMutex, portMAX_DELAY) == pdTRUE)
11             {
12                 /* Enviar comando a la cola */
13                 LED = 0;
14                 xQueueSend(xQueue, &LED, portMAX_DELAY);
15                 /* Liberar el mutex */
16                 xSemaphoreGive(xMutex);
17             }
18             xSemaphoreGive(yMutex);
19         }
20         vTaskDelay(2000);
21     }
22 }
23 /* Tarea 2 (Mayor Prioridad): Enviar comandos a la cola */
24 void vTask2(void *pv)
25 {
26     uint32_t LED ;
27     for(;;)
28     {
29         /* Toma el mutex y */
30         if (xSemaphoreTake(yMutex, portMAX_DELAY) == pdTRUE)
31         {
32             if (xSemaphoreTake(xMutex, portMAX_DELAY) == pdTRUE)
33             {
34                 /* Envía comando a la cola */
35                 LED = 1;
36                 xQueueSend(xQueue, &LED, portMAX_DELAY)
37                 /* Liberar el mutex */
38                 xSemaphoreGive(xMutex);
```

```
39     }
40     xSemaphoreGive(yMutex);
41 }
42 vTaskDelay(500);
43 }
44 }
45 /* Tarea de Control: Recibe comandos de la cola y controla el LED */
46 void vControlTask(void *pv)
47 {
48     uint32_t LED;
49     for(;;)
50     {
51         /* Recibir comandos de la cola */
52         if (xQueueReceive(xQueue, &command, portMAX_DELAY) == pdPASS)
53         {
54             /* Ejecutar el comando recibido */
55             if (LED == 1)
56             {
57                 /* Encender el LED */
58                 HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, GPIO_PIN_SET);
59                 printf("Control Task: LED ON\n");
60             }
61             else
62             {
63                 /* Apagar el LED */
64                 HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, GPIO_PIN_RESET);
65                 printf("Control Task: LED OFF\n");
66             }
67         }
68     }
69 }
```

En este programa se puede ver como ambas tareas terminarían estando en estado bloqueado a la espera de un mutex que se libera en otra tarea, debido a la relación tiempo de ejecución - prioridad entre las tareas.